

Beginner guide: Prometheus and Grafana for the llama.cpp RPC cluster

This section rewrites the monitoring part in a beginner-friendly way for the cluster described in the attached guide: NODE-A is the master running nginx, Open WebUI, and the HTTP inference endpoint, while NODE-B, NODE-C, and NODE-D are RPC workers on port 50052. [file:16] In this design, monitoring should focus on two things: the **host health of every node** and the **inference health of NODE-A**, because NODE-A is the only HTTP service that Prometheus can scrape directly for llama.cpp metrics.[file:16]

What Prometheus and Grafana are

Think of the stack this way:

- **Prometheus** is the collector and time-series database. It periodically visits a list of URLs called *targets* and saves the numbers it finds there.[web:48][web:63]
- **Grafana** is the viewer. It connects to Prometheus, lets you pick metrics, and turns them into dashboards, panels, and alerts.[web:61][web:48]
- **Exporters** are tiny services that expose system metrics in a format Prometheus understands. On Windows, the normal exporter is `windows_exporter`. [web:60][web:63]

A simple mental model is:

```
NODE metrics pages  --->  Prometheus  --->  Grafana dashboard
```

In your setup that becomes:

```
NODE-A/B/C/D windows_exporter  --->  Prometheus  --->  Grafana
NODE-A llama-server /metrics  --->  Prometheus  --->  Grafana
```

What gets monitored in your cluster

Your guide uses these nodes and roles:[file:16]

Node	Role	What to monitor	Why
NODE-A / 192.168.1.10 [file:16]	Master + nginx + Open WebUI + llama backend [file:16]	CPU, memory, and llama.cpp /metrics [file:16][web:35]	It is the main inference/control node. [file:16]
NODE-B / 192.168.1.11 [file:16]	RPC worker [file:16]	CPU and memory via windows_exporter [web:63]	Confirms worker health and load. [file:16]

Node	Role	What to monitor	Why
NODE-C / 192.168.1.12 [file:16]	RPC worker [file:16]	CPU and memory via windows_exporter [web:63]	Confirms worker health and load. [file:16]
NODE-D / 192.168.1.13 [file:16]	RPC worker [file:16]	CPU and memory via windows_exporter [web:63]	Confirms worker health and load. [file:16]

Prometheus will **scrape** these metrics pages every few seconds, and Grafana will display them as live charts.[web:48][web:61]

Step 1: Install windows_exporter on all four nodes

windows_exporter is the standard Windows agent that exposes CPU, memory, disk, and other host metrics to Prometheus.[web:63][web:60] Install it on NODE-A, NODE-B, NODE-C, and NODE-D so each machine has a metrics page, usually at port 9182.[web:63]

After install, each node should respond at a URL like these:[web:63]

- `http://192.168.1.10:9182/metrics` on NODE-A.[file:16][web:63]
- `http://192.168.1.11:9182/metrics` on NODE-B.[file:16][web:63]
- `http://192.168.1.12:9182/metrics` on NODE-C.[file:16][web:63]
- `http://192.168.1.13:9182/metrics` on NODE-D.[file:16][web:63]

When you open one of these URLs in a browser, it will look like raw text full of metric names and numbers. That is normal; Prometheus reads this text directly.[web:63]

Step 2: Enable llama.cpp metrics on NODE-A

The llama.cpp server can expose a Prometheus-compatible `/metrics` endpoint when metrics are enabled.[web:35][web:48] Since NODE-A is your master and the only HTTP inference endpoint in the guide, this is the machine Prometheus should scrape for llama.cpp request/queue/token metrics.[file:16]

Start llama-server on NODE-A with the normal RPC options from the guide and add `--metrics`: [file:16][web:35]

```
C:\llama.cpp\build\bin\Release\llama-server.exe ^
--model "C:\models\mistral-medium-3.5-q4_k_m.gguf" ^
--rpc "192.168.1.11:50052,192.168.1.12:50052,192.168.1.13:50052" ^
--host 0.0.0.0 --port 8080 ^
--ctx-size 8192 --n-predict 2048 ^
-ngl 70 -t 8 --parallel 2 --cont-batching ^
--metrics
```

Then test this URL from a browser on your LAN:[web:35][file:16]

- `http://192.168.1.10:8080/metrics`

If you see raw metrics text, that means Prometheus can scrape it later.[web:35]

Step 3: Install Prometheus

Prometheus is the service that stores all the metrics over time.[web:48] You can run it on NODE-A or on a separate machine, but for a home-lab setup it is simplest to run it on NODE-A since NODE-A already acts as the cluster control point in your guide.[file:16]

The most important Prometheus file is `prometheus.yml`. This file tells Prometheus **which URLs to scrape and how often**. [web:48]

A good starter config for your cluster is:

```
global:
  scrape_interval: 10s
  evaluation_interval: 10s

scrape_configs:
  - job_name: windows_nodes
    static_configs:
      - targets:
        - 192.168.1.10:9182
        - 192.168.1.11:9182
        - 192.168.1.12:9182
        - 192.168.1.13:9182

  - job_name: llamacpp_master
    metrics_path: /metrics
    static_configs:
      - targets:
        - 192.168.1.10:8080
```

What this means:[web:48]

- Every 10 seconds, Prometheus will visit all four Windows exporter targets.[web:63]
- Every 10 seconds, Prometheus will also visit `192.168.1.10:8080/metrics` for llama.cpp metrics.[web:35]
- It will store each metric as a time series so Grafana can draw charts later.[web:48]

After Prometheus starts, open its web UI and check **Status → Targets**. Each target should show as **UP**. [web:48] If a target is **DOWN**, that usually means the exporter is not running, the port is blocked by the firewall, or the URL/port is wrong.[web:63] [file:16]

Step 4: Learn the Prometheus “target” idea

This is the single most important Prometheus concept.

A **target** is just a URL that Prometheus scrapes. For your cluster, these are the targets:[file:16] [web:63] [web:35]

- `192.168.1.10:9182` → Windows metrics from NODE-A.[web:63]

- 192.168.1.11:9182 → Windows metrics from NODE-B.[web:63]
- 192.168.1.12:9182 → Windows metrics from NODE-C.[web:63]
- 192.168.1.13:9182 → Windows metrics from NODE-D.[web:63]
- 192.168.1.10:8080/metrics → llama.cpp metrics from the master.[web:35]

If these targets are up, Grafana has data to work with. If these targets are down, Grafana will show empty panels or “no data.”[web:48]

Step 5: Install Grafana

Grafana reads the data from Prometheus and turns it into dashboards.[web:61][web:48] After installing Grafana, you first add Prometheus as a data source.[web:61]

The basic first-time steps are:

1. Open Grafana in the browser.[web:61]
2. Go to **Connections** or **Data Sources** and add a new data source.[web:61]
3. Choose **Prometheus**. [web:61]
4. Set the Prometheus URL, usually something like `http://192.168.1.10:9090` if Prometheus runs on NODE-A.[web:48]
5. Click **Save & test** and confirm Grafana says the data source works.[web:61]

Once that succeeds, Grafana can query all metrics that Prometheus has scraped.[web:61]

Step 6: Start with a ready-made Windows dashboard

Grafana already has a Windows Exporter dashboard in its dashboard library, so you do not need to build every host-health chart yourself from scratch.[web:61] This is the easiest starting point for a beginner because it immediately gives you CPU, memory, disk, and network panels for the Windows nodes.[web:61]

So the recommended beginner path is:

- First import the **Windows Exporter Dashboard** from Grafana’s library.[web:61]
- Then create a second small custom dashboard just for llama.cpp metrics on NODE-A. [web:35][web:48]

That way, you learn Grafana in two stages instead of trying to do everything manually at once.[web:61]

Step 7: Understand what a Grafana dashboard and panel are

A **dashboard** is one page of charts. [web:61] A **panel** is one chart or stat box on that page. [web:61]

For example, you might build one dashboard called **llama Cluster Overview** with panels like these:[file:16][web:48]

- CPU usage by node.[web:63]

- Memory usage by node.[web:63]
- llama.cpp active requests on NODE-A.[web:35]
- llama.cpp deferred/queued requests on NODE-A.[web:35]
- Token throughput on NODE-A.[web:35][web:48]

In practice, adding a panel means:

1. Open a dashboard and click **Add visualization**. [web:61]
2. Choose your Prometheus data source. [web:61]
3. Enter a metric query in PromQL, which is Prometheus's query language. [web:48]
4. Pick a visualization type, usually **Time series** or **Stat**. [web:61]
5. Save the panel. [web:61]

Step 8: How to add things to a Grafana dashboard

Here is the beginner-friendly pattern for adding a panel:

Example A: Add a CPU chart for all nodes

1. In Grafana, open your dashboard and click **Add visualization**. [web:61]
2. Select the Prometheus data source. [web:61]
3. Enter a PromQL expression for CPU. The exact expression can vary by exporter version, but the idea is “show CPU use over time by instance.” [web:63] [web:48]
4. Pick **Time series** so each node becomes a separate line. [web:61]
5. Set the legend to show the instance label so you can tell which line belongs to which node. [web:61]
6. Save the panel as **CPU by node**. [web:61]

Example B: Add a memory panel

Do the same thing, but use a PromQL query based on total memory minus free memory from `windows_exporter` metrics. [web:63] Save it as **Memory used % by node**. [web:63] [web:61]

Example C: Add a llama.cpp request panel

Use your Prometheus data source again, but this time query the metric from NODE-A's `llama.cpp /metrics` endpoint. [web:35] [web:48] A good first panel is a **Stat** or **Time series** showing current active requests or queue/deferred requests on the master. [web:35]

The main idea is that **host panels** come from `windows_exporter` metrics, while **inference panels** come from `llama.cpp /metrics`. [web:63] [web:35]

Step 9: Suggested first dashboard layout

A good first custom dashboard for your cluster would be:

Row	Panel	Source
Top row	CPU by node [web:63]	Windows exporter metrics [web:63]
Top row	Memory by node [web:63]	Windows exporter metrics [web:63]
Middle row	Active llama requests [web:35]	NODE-A /metrics [web:35]
Middle row	Deferred/queued requests [web:35]	NODE-A /metrics [web:35]
Bottom row	Prompt tokens over time [web:35]	NODE-A /metrics [web:35]
Bottom row	Generated tokens over time [web:35]	NODE-A /metrics [web:35]

That dashboard answers the most useful beginner questions:[file:16] [web:35]

- Are all machines healthy? [web:63]
- Is NODE-A actively serving inference? [web:35]
- Is the queue backing up? [web:35]
- Is throughput normal? [web:48] [web:35]

Step 10: How to know if all nodes are being used

Prometheus and Grafana help you verify use, but they do not themselves distribute inference. In your architecture, `llama-server --rpc ...` on NODE-A is what uses all worker nodes. [file:16] `nginx` is only the reverse proxy layer in front of NODE-A's backend and should not be turned into a worker-node load balancer in this design.[file:16]

In practice, “all nodes are being used” usually means:

- NODE-A llama metrics show active work or throughput.[web:35]
- NODE-B/C/D show increased resource usage when generations are running.[web:63] [file:16]
- `rpc-server` and `llama-server` logs do not show repeated connection failures.[file:16]

So Grafana gives you visibility, but the actual multi-node behavior still comes from the `--rpc` list on NODE-A.[file:16]

Step 11: What to do when a panel is blank

If a panel shows No data, check in this order:

1. Is the exporter or service actually running on that node? [web:63] [file:16]
2. Does the target show UP in Prometheus Status → Targets? [web:48]
3. Can you open the raw metrics URL in a browser? [web:63] [web:35]
4. Did you pick the right Grafana data source? [web:61]

5. Is the PromQL query using the correct metric names for your exporter version? [web:63]

Most beginner problems come from the target not being reachable rather than from Grafana itself.[web:48][web:63]

Step 12: Simple operating workflow

A good daily workflow is:

1. Check Open WebUI works through nginx on NODE-A.[file:16]
2. Check Prometheus Targets and make sure all exporters plus NODE-A llama metrics are UP.[web:48]
3. Open Grafana and watch the Windows dashboard plus the llama dashboard.[web:61][web:48]
4. Run a test generation in Open WebUI and verify the graphs move on NODE-A and the worker nodes.[file:16][web:63][web:35]

That gives you a clean separation of roles:

- Open WebUI is for chatting.[file:16]
- Prometheus is for collecting metrics.[web:48]
- Grafana is for viewing dashboards.[web:61]
- llama.cpp on NODE-A is what actually coordinates the cluster through RPC.[file:16]